Assignment 3, The EM Algorithm

A: The EM-Algorithm for the t-Distribution

Carl-Emil Krarup Jensen

University of Copenhagen
Department of Mathematical Sciences

October 30, 2026

Agenda

- Theoretical background and implementation
- Numerical comparisons
- Benchmarking and profiling
- Discussion and conclusion

Theoretical background and implementation

Let $Y = (X, W) \in \mathbb{R} \times (0, \infty)$ have joint density

$$f(x, w) = \frac{1}{\sqrt{\pi\nu\sigma^2}\, 2^{(\nu+1)/2}\Gamma(\nu/2)} w^{\frac{\nu-1}{2}} e^{-\frac{w}{2}\left(1+\frac{(x-\mu)^2}{\nu\sigma^2}\right)}.$$

We can marginalize out w by using $\int_0^\infty w^{a-1}e^{-bw}\, dw = \Gamma(a)/b^a$, such that

$$\begin{aligned} f(x) &= \int_0^\infty f(x, w) dw \\ &= \frac{\Gamma((\nu+1)/2)}{\Gamma(\nu/2)\sqrt{\pi\nu\sigma^2}} \left(1 + \frac{(x-\mu)^2}{\nu\sigma^2}\right)^{-(\nu+1)/2} \end{aligned}$$

Which we recognize as $X = \mu + \sigma T_\nu$, where T_ν is a student's t-Distribution with ν degrees of freedom. The complete data log likelihood for n observations is

$$\begin{aligned} \ell_c(\mu, \sigma) = &-\frac{n}{2}\log(\pi\nu\sigma^2) - \frac{n(\nu+1)}{2}\log 2 - n\log\Gamma(\nu/2) \\ &+ \frac{\nu-1}{2}\sum_i \log w_i - \frac{1}{2}\sum_{i=1}^n w_i - \frac{1}{2\nu\sigma^2}\sum_{i=1}^n w_i(x_i-\mu)^2. \end{aligned}$$

The only terms depending on (μ, σ) are $-n\log\sigma^2/2 - \sum_i w_i(x_i-\mu)^2/(2\nu\sigma^2)$ depends on the unknown parameters.

Theoretical background and implementation

Differentiating wrt. μ and σ^2 and setting to 0 gives the complete data MLE.

$$\hat{\mu}_c = \frac{\sum_i w_i x_i}{\sum_i w_i}, \qquad \hat{\sigma}_c^2 = \frac{1}{n\nu} \sum_i w_i (x_i - \hat{\mu}_c)^2.$$

The weighted sum of squares is minimized at $\hat{\mu}_c$. maximizing over v then gives $\hat{v}_c$.

Let γ denote the pdf of the gamma distribution with shape parameter $\alpha > 0$, rate parameter $\beta > 0$;

$$\gamma(z) = \frac{\beta^\alpha}{\Gamma(\alpha)} z^{\alpha-1} \exp(-\beta x).$$

We can then recognize

$$W_i \mid X_i \sim \text{Gamma}\left(\underbrace{\frac{\nu+1}{2}}_{\text{shape}}, \underbrace{\frac{1 + (x_i - \mu)^2/(\nu\sigma^2)}{2}}_{\text{rate}} \right).$$

```
complete_data_mle <- function(x, w, nu) {
  joint_logdensity(x, w, 0, 1, nu) # validate the sample
  mu <- sum(w * x) / sum(w)
  v <- sum(w * (x - mu)^2) / (length(x) * nu)
  if (v <= 0) stop("No positive-scale maximizer")
  c(mu = mu, v = v)
}
```

Project structure

```
assignment_3
├── project_A
│   ├── complete_data.R
│   ├── em.R
│   ├── experiments.R
│   ├── fisher_information.R
│   ├── model.R
│   ├── optimization.R
│   └── simulation.R
├── report
└── tests/testthat
    ├── test-em.R
    ├── test-information.R
    ├── test-model.R
    └── test-optimization.R
```

Theoretical background and implementation

The E-step therefore replaces w_i by

$$\omega_i = E(W_i \mid x_i) = \frac{\nu + 1}{1 + (x_i - \mu)^2/(\nu v)}.$$

The M-step uses the complete-data formulas with these expected weights:

$$\mu^{(t+1)} = \frac{\sum_i \omega_i^{(t)} x_i}{\sum_i \omega_i^{(t)}},$$

$$v^{(t+1)} = \frac{\sum_i \omega_i^{(t)} (x_i - \mu^{(t+1)})^2}{n\nu}.$$

From `em.R`

```
 # E[W | x]: W|x is Gamma.
conditional_weights <- function(x, mu, v, nu) {
  validate_model(x, mu, v, nu)
  (nu + 1) / (1 + ((x - mu) / sqrt(nu * v))^2)
}

# One E-step followed by the analytical M-step.
em_step <- function(x, mu, v, nu)
complete_data_mle(x, conditional_weights(x, mu, v, nu), nu)


fit_em <- function(x, nu, start = c(median(x), log(IQR(x)^2 / 4)), tol =
        maxit = 10000, keep_history = TRUE, optimized = FALSE) {
  ...
  mu=eta[1], v=exp(eta[2]))
  converged <- FALSE
  for (iteration in seq_len(maxit)) {
    if (optimized) {
      w <- conditional_weights(x, eta[1], exp(eta[2]), nu)
      mu <- sum(w*x)/sum(w)
      estimate <- c(mu, sum(w*(x-mu)^2)/(length(x)*nu))
    } else estimate <- em_step(x, eta[1], exp(eta[2]), nu)
    candidate <- c(estimate[1], log(estimate[2]))
    new_ll <- t_loglik(x, candidate[1], exp(candidate[2]), nu)
    if (new_ll<ll-1e-10 * (1 + abs(ll))) stop("EM likelihood decreased")
    change<-abs(new_ll-ll)/(1+abs(ll)); eta<-candidate; ll<-new_ll
    history <- rbind(history, data.frame(iteration,
    if (score_error(x, eta, nu) <= tol && change <= tol) { converged <-
  }
  list(...)
}
```

Theoretical background and implementation

We test our implementation, and compare with Newton and BFGS. Both algorithms maximize the likelihood of the t-distribution inherited by X. Let $\eta = (\mu, \sigma^2)$, then the algorithms respectively update

```
test_that("E and M steps match a hand-calculated example", {
 # nu=2, mu=0, v=1: weights are 3,2,1 for residuals 0,1,2.
 x <- c(0,1,2); w <- c(3,2,1)
 expect_equal(conditional_weights(x,0,1,2),w)
 expect_equal(unname(em_step(x,0,1,2)),c(2/3,5/9))
})
```

1. $\eta_{t+1} \leftarrow \eta_t + \lambda_t \left(\nabla^2 \ell_x(\eta_t)\right)^{-1} \nabla \ell_x(\eta_t)$, λ_n is chosen by Armijo backtracking
2. $\eta_{t+1} \leftarrow \eta_t + a_t B_t \nabla \ell_x(\eta_t)$, where B_t is a Hessian approximation, and a_t is chosen by line search

In practice we use `optim()` for the BFGS implementation.

We test that the implementations compute similar parameter estimates for $\mu \in \{2, 5, 30\}$

```
test_that("three optimizers agree across fixed nu", {
 for(nu in c(2,5,30)) {
 x <- simulate_t_data(500,1,2,nu,seed=42+nu); f <- fit_all(x,nu)
 expect_true(all(sapply(f,`[[`,"converged")))
 expect_lt(diff(range(sapply(f,`[[`,"loglik"))),1e-7)
 expect_lt(diff(range(sapply(f,`[[`,"mu"))),1e-5)
 expect_lt(diff(range(sapply(f,`[[`,"v"))),1e-5)
 }
})
```

Numerical comparison

Define the scaled mean-score error $S(\mu, \log\sigma^2) = \max\left\{\frac{\sigma}{n}\left|\frac{\partial\ell}{\partial\mu}\right|, \frac{1}{n}\left|\frac{\partial\ell}{\partial\log\sigma^2}\right|\right\}$. With tolerance $\varepsilon = 10^{-7}$, convergence is declared when:

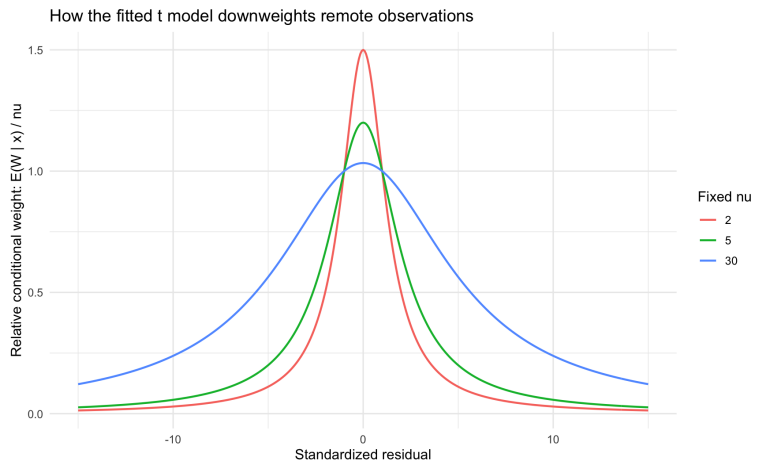
- **Newton:** $S(\mu, \log\sigma^2) \le \varepsilon$.
- **EM:** $S(\mu, \log\sigma^2) \le \varepsilon$ and $\frac{|\ell_{t+1}-\ell_t|}{1+|\ell_t|} \le \varepsilon$.
- **BFGS:** `optim()` reports successful termination using `reltol`$= 10^{-14}$, and the returned estimate satisfies $S(\mu, \log\sigma^2) \le \varepsilon$.

They share a final score-accuracy requirement, but use different internal stopping rules.

For a basic validation experiment we use $n = 1000$, $\mu = 1$, $\sigma^2 = 2$, $\nu = 5$, seed 405. All optimizers start at the sample median and squared half-IQR; $(m, s) = (median(x), IQR(x)/2)$.

Method	μ	σ^2	Seconds
EM	0.9589995	2.030235	0.029
BFGS	0.9589995	2.030236	0.009
Newton	0.9589995	2.030236	0.006

Validation experiment



Weights relative to ν for different observations.

Numerical comparison

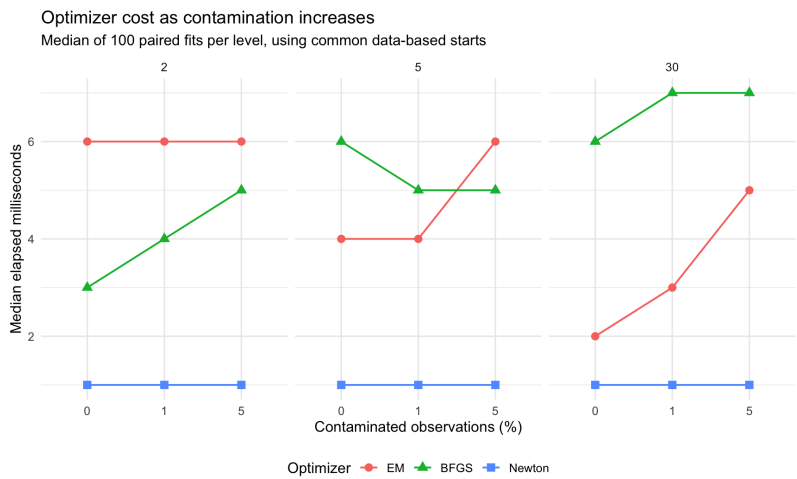
Does contamination change the speed or reliability of EM, BFGS and Newton?

We Generate 100 independent baseline samples of size $n = 500$ from t_5 with $\mu = 0$ and $\sigma = 1$. For each sample, randomly order observation positions and independently generate replacement values from $N(15, 1)$. Replace exactly 0, 5 or 25 observations.
We fit each resulting dataset under fixed $\nu = 2, 5, 30$ using all three optimizers. The clean generating model is always t_5; fitting a different $\nu 2$ represents a different tail assumption.
We run each experiment with two different initial guesses:

- The sample median and half-IQR scale (m, s)
- A poor initial guess 5 scale units away $(m + 5s, s/2)$

ν	Contam. (%)	All six certified (%)	Max. spread in ℓ
2	0	97	1.28×10^{-11}
2	1	96	1.33×10^{-11}
2	5	96	1.41×10^{-11}
5	0	99	8.98×10^{-12}
5	1	98	1.02×10^{-11}
5	5	98	1.18×10^{-11}
30	0	96	6.82×10^{-12}
30	1	89	8.53×10^{-12}
30	5	96	1.02×10^{-11}

Optimizer agreement over 100 datasets per scenario.



Median of each optimizer across contamination and ν values

Investigation of the Fisher information

Let $\lambda = \log(\sigma^2)$, $r = x - \mu$, $D = \nu\sigma^2 + r^2$, and $q = r^2/D$, with $\eta = (\mu, \lambda)$ and fixed ν.
We use the marginal distribution of X to compute the information matrix for one observation.

$$\begin{aligned}\ell_x(\eta) &= \log f(x;\eta) \\ &= C_\nu + \frac{\nu}{2}\lambda - \frac{\nu+1}{2}\log D, \qquad C_\nu \text{ constant in } \eta.\end{aligned}$$

$$\nabla_\eta \ell_x = \begin{pmatrix} \dfrac{(\nu+1)r}{D} \\ -\dfrac{1}{2} + \dfrac{\nu+1}{2}q \end{pmatrix}.$$

$$\nabla_\eta^2 \ell_x = \begin{pmatrix} \dfrac{(\nu+1)(r^2-\nu\sigma^2)}{D^2} & -\dfrac{\nu(\nu+1)\sigma^2 r}{D^2} \\ -\dfrac{\nu(\nu+1)\sigma^2 r}{D^2} & -\dfrac{\nu+1}{2}q(1-q) \end{pmatrix}.$$

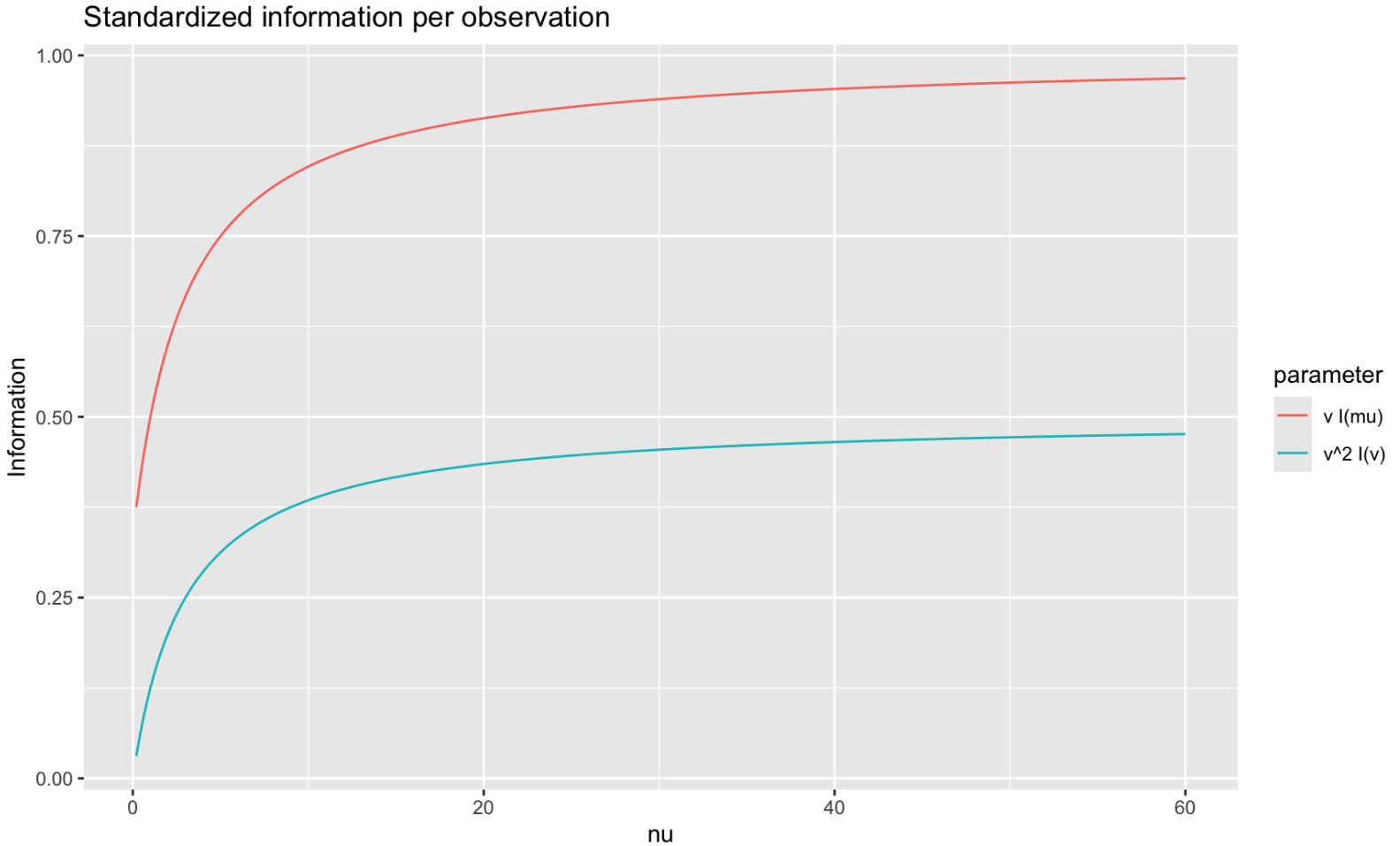
```
t_derivatives <- function(x, eta, nu) {
  mu<-eta[1]; v<-exp(eta[2]); validate_model(x,mu,v,nu)
  r <- x - mu; a<-nu * v; d<-a + r^2; q<-r^2 / d
  gradient<-c(sum((nu+1) * r / d), sum(-0.5+(nu+1)*q/2))
  cross <- sum(-(nu+1) * a * r / d^2)
  hessian<-matrix(c(sum((nu+1) * (r^2 - a) / d^2),
            cross, cross, sum(-(nu + 1)*q*(1-q)/2)),2)
  list(gradient = gradient, hessian = hessian)}
```

We can thus, express the Fisher information as

$$I(\eta) = -\mathbb{E}_\eta[\nabla_\eta^2 \ell_x] = \begin{pmatrix} \dfrac{\nu+1}{(\nu+3)\sigma^2} & 0 \\ 0 & \dfrac{\nu}{2(\nu+3)} \end{pmatrix}.$$

Investigation of the Fisher information

Heavy tails reduce information while limiting sensitivity to extremes



Information plotted against degrees of freedom

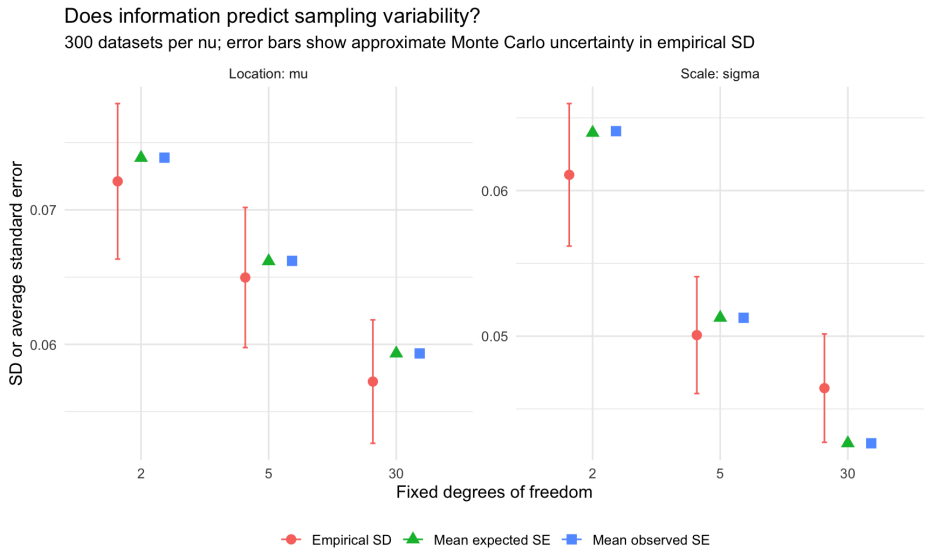
Investigation of the Fisher information

Experiment

- Generate 300 datasets per $\nu \in \{2, 5, 30\}$, each with $n = 300$, $\mu = 0$, $\sigma = 1$.
- Fit the correctly specified t_ν model.
- Compare empirical SD across datasets with average SE from inverse expected and observed information.

Main findings

- Expected and observed information give almost identical SEs.
- For μ, SE/SD is 1.019–1.037, with coverage 95–96%.
- For σ at $\nu = 30$, SE/SD is 0.918 and coverage is 93%: uncertainty is underestimated in this run.
- At 300 replications, coverage near 95% has Monte Carlo SE about 1.26 percentage points.



Error bars show approximate Monte Carlo uncertainty in the empirical SD, not confidence intervals for μ or σ.

Interpretation:
Fisher information broadly predicts sampling variability, but finite-sample calibration is imperfect.

Investigation of the Fisher information

ν	Parameter	Information	Mean SE	SE/SD	Coverage
2	μ	Expected	0.0739	1.0242	95.00%
2	μ	Observed	0.0739	1.0244	95.33%
5	μ	Expected	0.0662	1.0187	95.67%
5	μ	Observed	0.0662	1.0190	95.67%
30	μ	Expected	0.0593	1.0365	96.00%
30	μ	Observed	0.0593	1.0365	96.00%
2	σ	Expected	0.0640	1.0473	97.33%
2	σ	Observed	0.0641	1.0491	97.00%
5	σ	Expected	0.0513	1.0238	95.67%
5	σ	Observed	0.0513	1.0237	95.67%
30	σ	Expected	0.0426	0.9184	93.00%
30	σ	Observed	0.0426	0.9184	93.00%

SE/SD is the average estimated SE divided by the empirical SD. Values near one indicate agreement. Coverage is the proportion of nominal 95% Wald intervals containing the true parameter.

Expected and observed information give similar results. The largest calibration discrepancy occurs for σ at $\nu = 30$, where SE/SD is approximately 0.918 and coverage is 93%.

Computational complexity

All three algorithms are linear in the sample size. Thus, their asymptotic complexity is similar.

Let b_t be the number of likelihood iterations during line search t.

Method	Complexity per iteration	Computations per iteration
EM	$O(n)$	Weights, weighted sums, ℓ and $\nabla\ell$
Newton SGD	$O(n(t+b_t))$	$\nabla\ell$ and $\nabla^2\ell$, solve 2×2 system, line search
BFGS	$O(n(t+b_t))$	Gradient, 2×2 $\nabla^2\ell$ approx update, line search

For T iterations the costs are

1. EM: $O(nT)$
2. Newton/BFGS $O\left(n\left(T+\sum_{t=1}^{T} b_t\right)\right)$

However in practice

- EM can require many iterations.
- Newton can converge in very few iterations near the solution.
- BFGS circumvents the analytical Hessian but may use more iterations.

Benchmarking and profiling

We compare the computational effort of the algorithms.

Throughout all experiments the stopping criterion for EM and Newton is $\max\left\{\frac{\sigma\partial_\mu \ell}{n}, \frac{\partial_\lambda \ell}{n}\right\} < 10^{-7}$. BFGS uses the built in stopping rule stopping the algorithm when we $-\ell$ can't be reduced by more than $\varepsilon(|-\ell| + \varepsilon)$, where we set $\varepsilon = 10^{-14}$, and require that BFGS meets the same score criteria as the other algorithms[1].
We examine $\nu = \{1, 5, 30\}$ for $n = 1000$ in each experiment. In all experiments $(\mu, \sigma^2) = (1, 2)$. Experiments are repeated 7 times for each ν

[1] See [R Core Team, 2026] documentation under details for `reltol`.

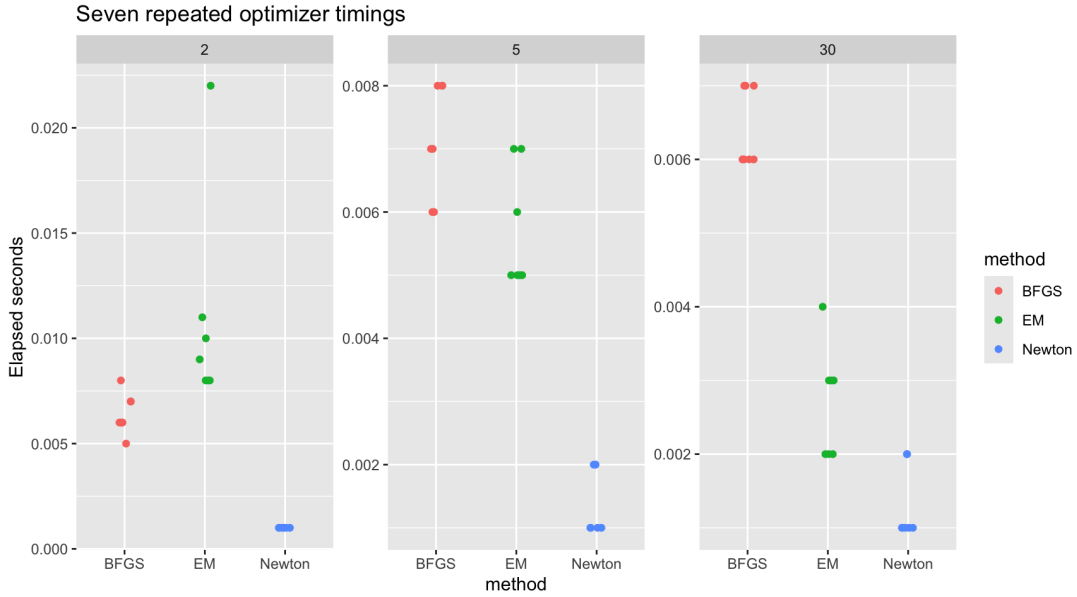
Benchmarking and profiling

Computational effort and median runtime

ν	Method	Iter.	ℓ eval.	ms
2	EM	28	29	9
2	BFGS	–	28	6
2	Newton	3	4	1
5	EM	16	17	5
5	BFGS	–	30	7
5	Newton	4	5	1
30	EM	7	8	3
30	BFGS	–	28	6
30	Newton	4	5	1

BFGS accepted iteration counts are unavailable from `optim()`.

With only two parameters, analytical derivatives and a small Hessian make Newton inexpensive.



- All experiments passed the accuracy check. Methods agree on estimates and final likelihood to numerical precision.
- Newton has the lowest median runtime for every tested ν.
- EM requires fewer iterations as ν increases: 28, 16, then 7.
- BFGS uses substantially more likelihood evaluations than Newton.

Caveat: Millisecond-scale runtimes are machine-dependent.

Benchmarking and profiling

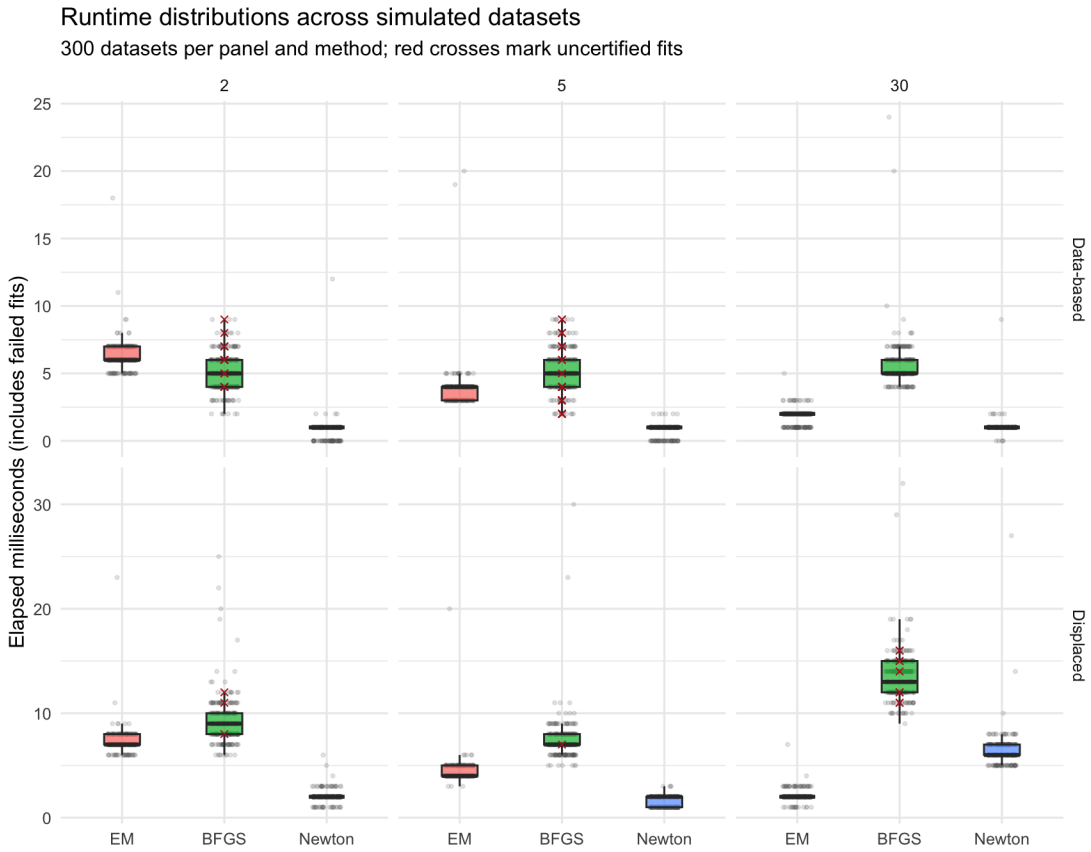
We now generate 300 datasets of size $n = 300$ for each $\nu \in \{2, 5, 30\}$, with $\mu = 0$ and $\sigma = 1$. We fit the three algorithms from two shared starting points like earlier

- The sample median and half-IQR scale (m, s)
- A poor initial guess 5 scale units away $(m + 5s, s/2)$

In total we have

$$300 \times 3 \times 2 \times 3 = 5400 \text{ fits.}$$

Reading the figure: Boxes show medians and interquartile ranges. Timings include failed fits. One timing per fit combines dataset difficulty with machine timing noise.



- Newton has the lowest typical runtime in most scenarios.
- Displaced starts generally increases runtime. Sensitivity depends on both method and ν.
- Red crosses identify uncertified fits.
- Newton runtimes get dispersed as initial parameters are displaced.

Benchmarking and profiling

We profile the EM implementation

```
profile_em <- function(n=20000, repetitions=25,
                       interval=0.002) {
  x <- simulate_t_data(n,1,2,2,seed=188)
  path <- tempfile()
  on.exit(unlink(path))

  Rprof(path,interval=interval)
  tryCatch(
    for(i in seq_len(repetitions)) fit_em(x,2),
    finally=Rprof(NULL)
  )
  profile <- summaryRprof(path)

  rows <- list()
  for(i in seq_len(7)) {
    for(fast in c(FALSE,TRUE)) {
      f <- fit_em(x,2,optimized=fast)
      rows[[length(rows)+1]] <- data.frame(
        optimized=fast, seconds=f$elapsed,
        mu=f$mu, v=f$v, loglik=f$loglik,
        converged=f$converged
      )
    }
  }
  list(profile=profile,
       comparison=do.call(rbind,rows),
       n=n, repetitions=repetitions)
}
```

Setup

- 25 readable EM fits on one fixed sample.
- $n = 20{,}000$, $\mu = 1$, $\sigma^2 = 2$, $\nu = 2$.
- Sampling interval 2 ms.
- After profiling, time each implementation seven times without `Rprof()`.

Function	Total %	Self %
`proc.time`	55.32	55.32
`complete_data_mle`	16.49	1.46
`t_derivatives`	13.99	7.72
`joint_logdensity`	13.15	2.09
`t_loglik`	12.11	0.42
`dt`	10.44	10.44

- Timing instrumentation dominates: `proc.time` accounts for 55.32% of sampled time[a].
- The M-step evaluates the joint density for validation, although its update only needs weighted sums.
- The streamlined version removes this unused density evaluation.
- Marginal likelihood and derivative calculations also contribute.

[a] Total time includes called functions. Self time excludes them.

Benchmarking and profiling

Setting `optimized=fast`:

The comparison loop sets `fast` to `FALSE`, then `TRUE`. Inside `fit_em()`, the branches are:

```
if (optimized) {
  w <- conditional_weights(
    x, eta[1], exp(eta[2]), nu
  )
  mu <- sum(w*x)/sum(w)
  estimate <- c(
    mu,
    sum(w*(x-mu)^2)/(length(x)*nu)
  )
} else {
  estimate <- em_step(
    x, eta[1], exp(eta[2]), nu
  )
}
```

Setting `optimized=FALSE`: `em_step()` calls `complete_data_mle()`, which evaluates the complete joint density to validate its inputs.

Setting `optimized=TRUE`: Computes the same conditional weights and weighted M-step directly, avoiding that unused density evaluation.

Both branches retain likelihood monitoring, score checks and timing instrumentation.

Profiling motivates the change

Function	Self time (%)
`proc.time`	53.09
`dt`	11.18
`t_derivatives`	9.38
`dnorm`	4.79
`dchisq`	2.40

The streamlined branch removes the joint-density work involving `dnorm()` and `dchisq()`.

Implementation	Median runtime
Readable EM	50 ms
Streamlined EM	36 ms
Speedup	$50/36 \approx 1.39$

Seven runs per implementation, $n = 20{,}000$, $\nu = 2$.
Both implementations converge. Location and scale-squared differences are below 10^{-12}, and likelihood differences below 10^{-9}.

Discussion and conclusion

Within most of the settings explored Newtons algorithm performs better than the EM and the BFGS algorithm in terms of runtime.

Expanding the study to settings where latent variables makes the log likelihood simple, but marginal optimization hard will exhibit the strengths of the EM algorithm.

Exploring settings with expensive Hessians or indefinite and gradients will also showcase how the EM algorithm can ease computation and provide a tool for robust optimization.

Further studies within this framework could include

- Removing timing/history overhead and unnecessary Hessian calculations.
- Plotting runtime against achieved accuracy over several tolerances.
- Study sample size sensitivity.
- Exploring different parameter configurations.
- Testing alternative step size logic's.

References I



R Core Team (2026).

optim: General-purpose Optimization.

R Foundation for Statistical Computing.

R documentation, stats package. Accessed 2026-10-30.